

Меню **НА ЭТОЙ СТРАНИЦЕ**

[Установка](#)

[Настройка и аутентификация](#)

[Ваш первый запрос](#)

[Мультимодальные входы](#)

[Задачи](#)

[Параметры клиента](#)

[Ошибки](#)

[Следующие шаги](#)

SDK Interfaze

[скопировать в буфер обмена](#)

Официальные SDK Interfaze – это самый быстрый способ взаимодействия с Interfaze. Они соответствуют стандарту Chat Completion API, так что интерфейс знаком, а поверх добавлены специфичные для Interfaze элементы: `precontext`, `reasoning`, задачи и ограничения в виде типизированных полей первого класса.

- [TypeScript / JavaScript SDK](#)
- [Python SDK](#)

Установка

typescript 

npm / yarn

```
npm install interfaze
# or
yarn add interfaze
```



Настройка и аутентификация

Клиент считывает `INTERFAZE_API_KEY` из окружения, поэтому его можно создать без аргументов. Получите свой ключ API на [панели управления](#).

typescript ▾

SDK Interfaze

```
import { Interfaze } from "interfaze";

const interfaze = new Interfaze(); // or new Interfaze({ apiKey: "<your-api-key>" })
```

- Не нужно настраивать базовый URL или имя модели – они встроены.
- В Python `AsyncInterfaze` – это идентичный асинхронный клиент, в котором каждый вызов является `await`-able.

Ваш первый запрос

typescript ▾

SDK Interfaze

```
import { Interfaze, responseFormat } from "interfaze";
import { z } from "zod";

const interfaze = new Interfaze();

const IDSchema = z.object({
  first_name: z.string().describe("First name on the ID"),
  last_name: z.string().describe("Last name on the ID"),
  dob: z.string().describe("Date of birth on the ID"),
  driver_licence_number: z.string().describe("Driver licence number on the ID"),
});

const response = await interfaze.chat.completions.create({
  messages: [
```

- `response_format()` нормализует JSON-схему для `Interfaze`. В TypeScript передайте схему `Zod` через `z.toJSONSchema()`.
- `message.content` – это строка JSON, поэтому ее нужно проанализировать. Оставьте корень схемы в виде `object`, так как корень, не являющийся объектом, оборачивается в ключ `result`.
- `precontext` содержит необработанные метаданные ответа, такие как ограничивающие рамки и показатели достоверности. Подробнее о [преконтексте](#).

Мультимодальные входные данные

`inputs.*` создает для вас части контента и преобразует необработанные байты или локальный файл в URI данных.

typescript ▾

SDK Interfaze

```
import { inputs } from "interfaze";

inputs.image("https://r2public.jigsawstack.com/interfaze/examples/id.jpg"); // i
inputs.file("https://arxiv.org/pdf/1706.03762"); // file part
inputs.audio("https://r2public.jigsawstack.com/interfaze/examples/stt_medical_st
inputs.image(await inputs.fromPath("./photo.png")); // read a local file (Node c
inputs.file(await inputs.dataUrl(pdfBytes, "application/pdf"), { filename: "repe
```

Подробнее о [работе с файлами](#).

Tasks

`tasks.*` запускает один встроенный инструмент вместо полной модели, что быстрее и дешевле. Каждый вспомогательный инструмент принимает исходный код и возвращает необработанный результат.

typescript ▾

SDK Interfaze

```
await interfaze.tasks.ocr(url);
await interfaze.tasks.objectDetection(url);
await interfaze.tasks.guiDetection(url);
await interfaze.tasks.webSearch(query);
await interfaze.tasks.scrape(url);
await interfaze.tasks.transcribe(url);
await interfaze.tasks.translate(text, { to: "Spanish" });
await interfaze.tasks.forecast(csvUrl, { periods: 30, unit: "days" });
```

Подробнее о [запуске задачи](#).

Параметры клиента

Маршрутизатор, кэш и параметры потоковой передачи настраиваются один раз на клиенте.

typescript ▾

```
const interfaze = new Interfaze({
  showAdditionalInfo: true, // stream <precontext> deltas as they're produced
  bypassMoA: true, // skip the mixture-of-architecture router
  bypassCache: true, // skip the semantic cache
});
```

Learn more about [caching](#) and [bypassing MoA](#).

Errors

Every error class is exported so you can catch and narrow on it. `InterfazeError` covers client-side problems like a missing key or an invalid guard code. Everything else is an `APIError` subclass carrying the status and code, such as `BadRequestError` (400), `AuthenticationError` (401), and `RateLimitError` (429).

typescript ▾

Interfaze SDK

```
import { BadRequestError, InterfazeError, RateLimitError } from "interfaze";

try {
  await interfaze.chat.completions.create({ messages: [{ role: "user", content
} catch (error) {
  if (error instanceof RateLimitError) console.error("Slow down:", error.statu
  else throw error;
}
```

Next steps

- [Structured outputs](#)
- [Streaming](#)
- [Reasoning](#)
- [Function calling](#)
- [Guardrails](#)
- [Chat Completion API reference](#)

< Previous
Postgres LLM

Interfaze

PRODUCT

- Playground
- OCR
- Models
- Leaderboards
- Pricing

CAPABILITIES

- Vision
- Web
- Audio
- GUI
- Compute

DEVELOPERS

- Docs
- API Ref
- Structured Outputs
- Integration

RESOURCES

- Blog
- FAQs
- Help

COMPANY

- Careers
- Sales
- Status

LEGAL

- Privacy
- Terms
- DPA